

Machine Learning Quote

“Machine Learning won't replace humans, but those who understand it will.”

Meaning:

ML will not replace thinking humans, but people who understand ML will replace those who don't.

Goal of This Chapter

To classify data using different algorithms.

What is Classification?

A classification algorithm is a supervised learning method used to predict a category or class label.

Examples:

- Cat vs Dog
- Spam vs Not Spam

. Obese vs Not Obese

Weight	Obesity
26kg	No
39kg	No
48kg	No
96kg	Yes
128kg	Yes
82kg	Yes

Model learns pattern:

Higher weight → more obesity probability

Key Idea:

Classification means learning boundaries between categories.

Linear Regression vs Logistic Regression

Linear Regression

Used for: Continuous values (numbers)

Example:

- Salary prediction
- House price prediction

Logistic Regression

Used for:

Classification (0 or 1, Yes/No)

Example:

- Obese or not
- Spam or not

Core Difference:

Linear Regression Logistic Regression

Predicts numbers Predicts probability

Straight line S-shaped curve

Linear Regression Logistic Regression

Can go <0 or >1 Stays between 0 and 1

Logistic Regression

Problem with Linear Regression for classification:

- Outputs can be negative ✕
- Outputs can be >1 ✕
- Not interpretable as probability ✕
- **Solution:**
 - We “squash” values using sigmoid function.

Sigmoid Function:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Meaning:

- Converts any number into range: **0 to 1**

- Interpreted as probability

Why e is used?

- $e = 2.71828$
- Natural growth constant
- Gives smooth curve
- Comes from probability + calculus

Decision Rule:

- If output $> 0.5 \rightarrow$ Class 1 (Yes)
- If output $< 0.5 \rightarrow$ Class 0 (No)
- **Intuition:**
- Logistic regression is linear regression + sigmoid function

Linear vs Logistic Regression

Linear:

- uses cost function (MSE)
- fits straight line

Logistic:

- uses log loss
- fits probability curve

Log Loss (Binary Cross Entropy)

Measures how wrong predicted probabilities are.

Formula:

$$-\frac{1}{m} \sum [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Meaning:

- If prediction is correct → low loss
- If wrong with high confidence → high loss

Intuition:

Prediction	Actual	Loss
0.99	1	very low

Prediction	Actual	Loss
0.01	1	very high

Why log?

- punishes confident wrong predictions heavily
- works with probabilities

Confusion Matrix

Used to evaluate classification.

Structure:

	Predicted Yes	Predicted No	
Actual Yes	TP	FN	
Actual No	FP	TN	

☐ TP = True Positive

☐ TN = True Negative

☐ FP = False Positive

☐ FN = False Negative

Evaluation Metrics

Accuracy

$$\frac{TP + TN}{TP + TN + FP + FN}$$

✓ Overall correctness

Precision

$$\frac{TP}{TP + FP}$$

✓ Of predicted positives, how many are correct

Use when FP is dangerous (spam detection)

Recall

$$\frac{TP}{TP + FN}$$

✓ Of actual positives, how many we caught

Use when FN is dangerous (disease detection)

F1 Score

$$2 \times \frac{Precision \times Recall}{Precision + Recall}$$

✓ Balance between precision & recall

K-Nearest Neighbors (KNN)

A lazy learning algorithm that classifies based on nearest points.

Steps:

1. Choose K
2. Calculate distance (Euclidean)
3. Find nearest neighbors
4. Majority vote

Formula (distance):

$$(x_2 - x_1)^2 + (y_2 - y_1)^2$$

Notes:

- K should be odd
- Works in multi-dimensional space
- Can also be used for regression (average of neighbors)

Intuition:

“Tell me your neighbors, I’ll tell who you are.”

Naive Bayes

A probabilistic classifier based on Bayes theorem.

Key Idea:

Assumes:

All features are independent (naive assumption)

Independence:

Coin toss:

- $P(H) = 1/2$

Cards:

- dependent events

Bayes Theorem:

$$P(A | B) = \frac{P(A) \cdot P(B | A)}{P(B)}$$

Meaning:

Probability of A given B

ML Form:

$$P(y | x_1, x_2, \dots) = \frac{P(y) \cdot P(x | y)}{P(x)}$$

Intuition:

“What is the probability of class given features?”

A tree-like model that makes decisions step by step.

Decision Tree

A tree-like model that makes decisions step by step.

Structure:

- Root node → first split
- Branch → condition path
- Leaf → final output

money = ?

if money < 20:

snack

elif money > 20 and <60:

fast food

elif money > 60 and < 90:

dinnear

else:

5-star

How root is selected?

Using:

Entropy + Information Gain

Entropy:

$$H(S) = -\sum p_i \log_2(p_i)$$

Information Gain:

$$IG = Entropy(parent) \\ - Entropy(children)$$

Logic:

Best split = highest information gain

Outlook	Temp	Humidity	Wind	Play
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	False	Yes
Rain	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rain	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rain	Mild	High	True	No

Outlook => summary(2Y,3N), overcast(4Y,0N),
rain(3Y,2N)

Sunny => hot(2N,0Y), mild, cool

How are you choosing outlook as a root
node?

so firstly, Outlook is a root node, overcast is a
leaf, and the entire game is called branches.

Okay, so we use entropy and information gain to choose the root node. Since we have 5 columns (outlook, temperature, humidity, wind, play), we first implement the formulas for each column based on the highest information gain, and we use that column as the root node.

Steps:

1. Choose the best feature
2. Split data
3. Repeat
4. Stop when:
 - pure nodes
 - max depth

Support Vector Machine (SVM)

A classifier that finds best boundary separating classes.

Key Idea:

We create:

- hyperplane
- margin
- support vectors

Equation:

$$wx + b = 0$$

Decision rule:

- $wx + b > 0 \rightarrow \text{Class 1}$
- $wx + b < 0 \rightarrow \text{Class 2}$

Concepts:

Hyperplane:

Decision boundary separating classes

Margin:

Distance between hyperplane and nearest points

Support Vectors:

Points closest to the boundary

Kernels:

Used when data is not linearly separable.

They transform data into higher dimensions.

Intuition:

“Find the widest possible separation between classes”

Now connect everything:

Classification Problem



We need models to classify data



Use:

- Logistic Regression
- KNN
- Decision Tree
- Naive Bayes
- SVM



Each model learns differently:

- probability (Logistic, Naive Bayes)

- . distance (KNN)**
- . rules (Decision Tree)**
- . margin separation (SVM)**



The model makes predictions



We evaluate using:

- . Accuracy**
- . Precision**
- . Recall**
- . F1 Score**
- . Confusion Matrix**



We improve the model using:

- . Cross Validation (K-Fold)**

Classification is the process of teaching machines to assign categories using algorithms such as Logistic Regression, KNN, Decision Trees, Naive Bayes, and SVM. We evaluate their performance using metrics like accuracy, precision, recall, and F1-score to ensure the model generalizes well on unseen data.